Agent 分层记忆系统设计文档

面向分布式 Agent Runtime 的架构设计、实施方案与时序/流程图

版本	v1.0
文档类型	架构设计 / 实施方案 / 落地路线图
适用对象	分布式 Agent、长期任务执行系统、多 Agent 协同系统
核心目标	解决上下文爆炸、状态丢失、任务不可接管与检索失控

一句话总结： 这个系统不是“保存聊天记录”的数据库，而是一个日志驱动、状态落地、混合检索、可接管恢复的 Agent 记忆运行时。

目录

- 1. 文档目标与设计范围
- 2. 核心问题定义
- 3. 设计原则
- 4. 总体架构
- 5. 分层记忆模型
- 6. 底层存储结构映射
- 7. 核心对象模型
- 8. 写入方案
- 9. 回载方案
- 10. 详细实施方案
- 11. 实现步骤与分阶段计划
- 12. 工作时序图
- 13. 流程图
- 14. 可观测性与运维
- 15. 风险与边界
- 16. 最终落地建议

1. 文档目标与设计范围

本文档将前面讨论的内容收敛为一套可工程化的 Agent 记忆系统设计方案。该系统不是“聊天记录存储器”，而是一个面向 Agent Runtime 的分层记忆子系统，它需要同时承担工作记忆装配、长期状态落地、过程事件回放、checkpoint 恢复、结构化与语义化检索、多 Agent 接管与协同、优先级与降级执行支撑。

本文档输出包含：

- 完整文档化说明
- 实现步骤
- 具体详细实施方案
- 工作时序图与流程图

2. 核心问题定义

2.1 当前 Agent 系统常见痛点

- 上下文爆炸：会话变长、项目变大、工具输出增多后，prompt 迅速膨胀，导致成本上升、相关信息被淹没、长任务推理质量下降。
- 多 Agent 不可观测：子 Agent 卡住、重试、超时或异常退出时，父 Agent 往往只能感知“没完成”，无法知道具体阻塞点。
- 执行进度丢失不可恢复：关键执行状态隐含在当前上下文内，一旦 Agent 重启、模型切换、节点失效，任务无法从中间状态继续。
- 级联阻塞无法保护高优先级任务：支线任务长时间阻塞时，系统无法自动熔断非关键任务、释放资源、保证主线任务继续推进。

3. 设计原则

原则	说明
状态外化	所有关键执行状态必须写入系统，而不是只存在于当前模型上下文中。
事件优先	关键动作先写事件流，再写状态真相，确保过程可回放、故障可审计。
热冷分层	区分工作记忆、热状态、持久状态、历史事件与检索索引。
小而准	L0 工作记忆尽量小，但必须覆盖目标、证据、约束与失败原因。
混合检索	综合结构化过滤、倒排、向量和关系扩展，避免单一路径失准。
可接管	恢复与接管依赖事件流、checkpoint、当前真相与可回载记忆。

4. 总体架构

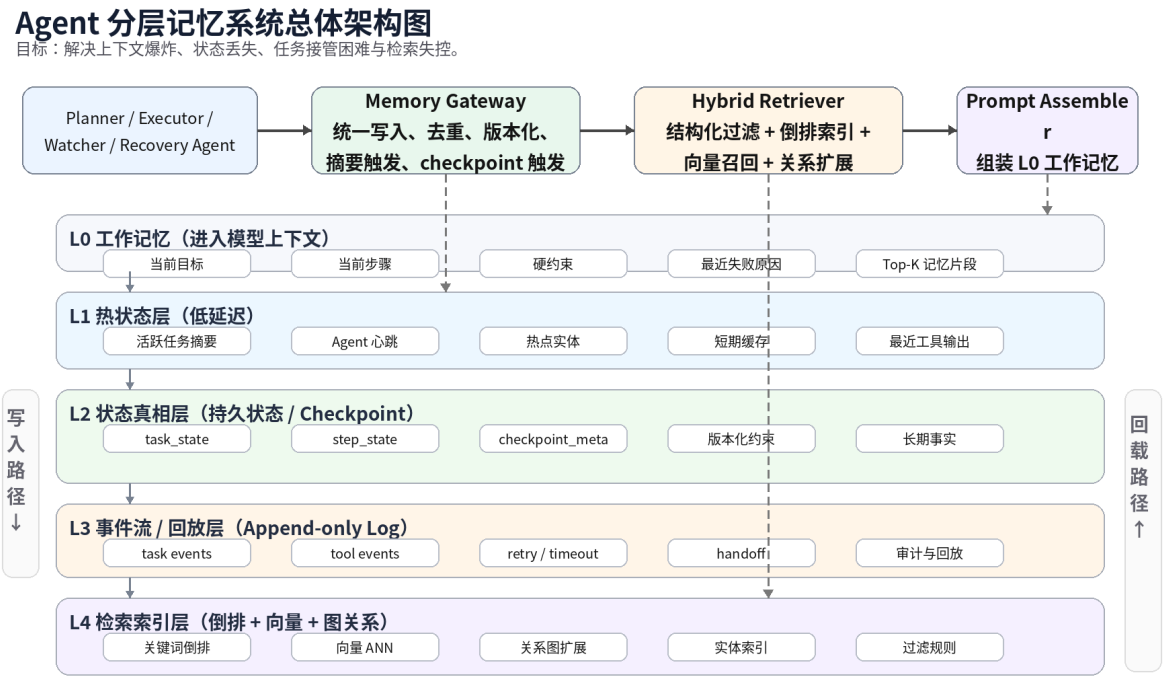


图 1：Agent 分层记忆系统总体架构图

层级	名称	主要内容	主要目标	是否直接进 Prompt
L0	工作记忆	当前目标、当前步骤、硬约束、最近失败原因、Top-K 证据	为模型本轮推理服务	是
L1	热状态层	活跃任务摘要、Agent 心跳、热点实体、最近工具输出、短期缓存	低延迟读写	否
L2	状态真相层	task_state、step_state、checkpoint_meta、版本约束、长期事实	恢复、接管、审计	否
L3	事件流 / 回放层	task events、tool events、retry / timeout、handoff、审计记录	全过程回放	否
L4	检索索引层	倒排索引、向量索引、关系图扩展、过滤规则	按需回载	间接

5. 分层记忆模型

“记忆爆炸”不是单纯容量问题，而是工作记忆过大、状态更新过频繁、历史事件不断累积、检索找不回真正相关内容四类问题叠加。因此系统必须区分过程、当前真相、检索入口与真正进入 prompt 的最小证据集合。

6. 底层存储结构映射

存储思想	最适合解决的问题	在本系统中的作用
Append-only Log	进度不丢、过程可回放、故障接管	L3 事件流
B+ 树 / 聚簇索引	当前状态真相、checkpoint 元数据、版本状态	L2 状态真相层
LSM-tree	高频写入、冷热下沉、写放大优化	高写入 checkpoint / 状态更新场景
倒排索引	关键词、规则、实体名、错误码精确召回	L4 精确检索
向量索引	相似案例、语义相关片段	L4 语义召回
图结构	依赖图、因果链、任务关系、实体关系	L4 关系扩展

7. 核心对象模型

对象	说明	核心字段
Task	一个可调度任务	task_id, parent_task_id, priority, deadline, state
Step	Task 内的执行步骤	step_id, task_id, agent_id, status, retry_count
Checkpoint	可恢复快照	checkpoint_id, task_id, step_id, version, input_ref, output_ref
Event	append-only 事件	event_id, trace_id, event_type, task_id, step_id, payload
MemoryChunk	可检索记忆块	chunk_id, scope, summary, source_ref, importance
Constraint	约束项	constraint_id, scope, version, text, hard_or_soft

Entity / Relation	实体与关系边	entity_id, relation_type, weight, src_id, dst_id

8. 写入方案

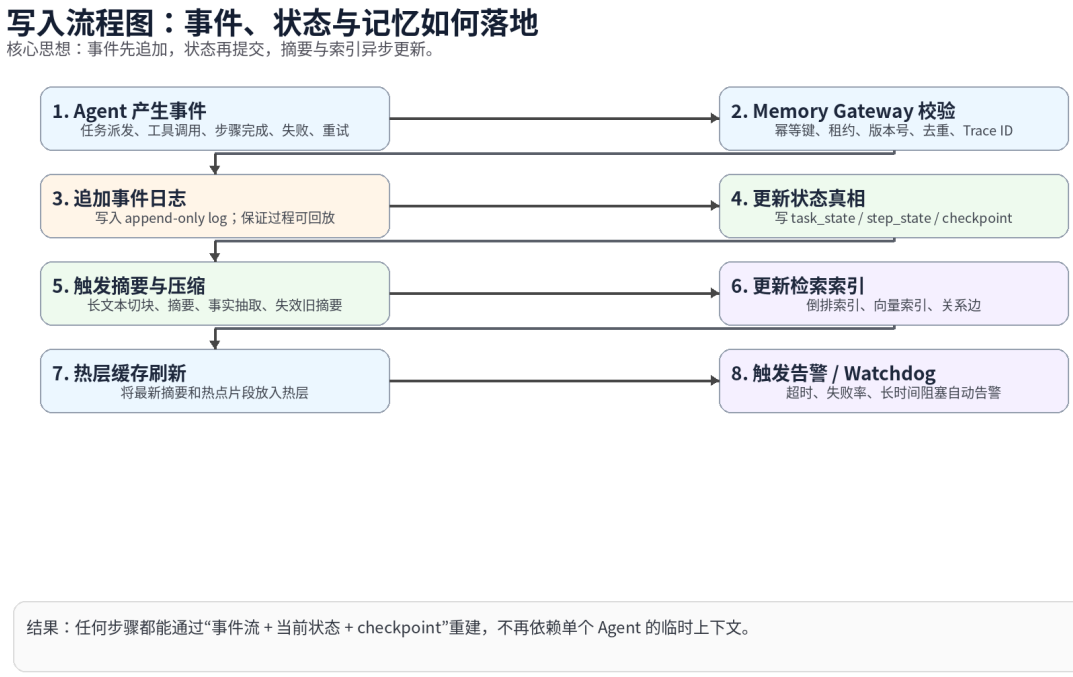
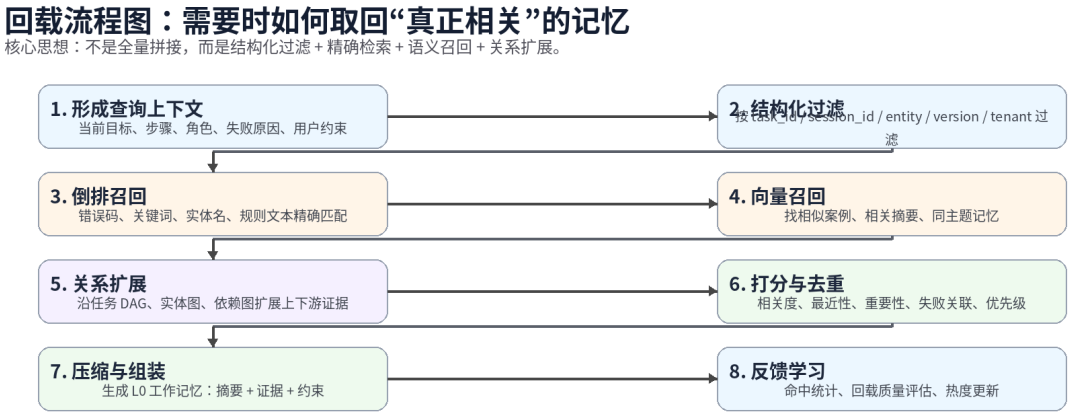


图 2：写入流程图

触发点	写事件	写状态	写 Checkpoint	补充动作
task created	是	是	否	初始化 task_state = queued
step started	是	是	否	记录租约、开始时间
tool result small	是	是	视情况	轻量结果可仅更新摘要
tool result large	是	是	是	长文本分块、摘要、抽取事实
step completed	是	是	是	刷新热层、更新命中热度
retry / timeout	是	是	是	触发 Watchdog 与恢复逻辑
handoff	是	是	是	写 handoff payload 与接管链路

9. 回载方案



结果：工作记忆始终保持“小而准”，记忆系统关注的是“回载需要的信息”，而不是“保存所有原文进 prompt”。

图 3：回载流程图

- 形成查询上下文：当前目标、步骤、角色、最近失败原因、用户约束、任务优先级与依赖链。
- 结构化过滤：按 session_id / task_id / entity_id / version / tenant 范围裁剪。
- 精确召回：利用倒排层查找错误码、规则条目、实体名、工具名与文本约束。
- 语义召回：寻找相似步骤摘要、相似失败案例、相似工具输出和同主题记忆。
- 关系扩展：沿任务 DAG、实体图、依赖图补充上下游证据。
- 统一打分去重：相关度、最近性、重要性、失败关联、优先级、可信度。
- 压缩组装：生成 L0 工作记忆。

10. 详细实施方案

子系统	职责	输出
Memory Gateway	统一写入、幂等、版本校验	事件、状态、摘要任务
Event Logger	事件持久化	append-only event log
State Store	保存当前状态真相	task/step/checkpoint 状态
Summarizer / Compactor	长文本压缩、事实抽取	summary / facts / chunk
Indexer	更新倒排、向量、关系索引	queryable index

Retriever	混合检索	Top-K memory set
Prompt Assembler	组装 L0 工作记忆	prompt context
Watchdog	超时、阻塞、失败率检测	告警 / 重试 / 接管触发
Recovery Manager	接管与恢复	恢复任务与 handoff 记录

- Checkpoint 至少应包含：checkpoint_id、task_id、step_id、plan_version、constraint_version、last_event_offset、input_refs、output_refs、retry_context、next_actions、handoff_ready_payload。
- 回载统一打分建议：MemoryScore = semantic_relevance + keyword_match + recency + importance + failure_association + dependency_weight + trust_score（可按业务加权）。

11. 实现步骤与分阶段计划

阶段	目标	关键交付
阶段 0	统一对象模型与状态机	Task / Step / Event / Checkpoint schema，状态枚举，trace_id 规则
阶段 1	先解决“进度不丢”	event log、state store、checkpoint、watchdog 基础版
阶段 2	先解决“上下文不能全塞”	分块、摘要器、倒排索引、向量索引、Top-K 回载接口
阶段 3	热冷分层	热层缓存、热度更新、冷层归档、失效摘要清理
阶段 4	接管与恢复	handoff payload、lease / heartbeat、timeout detection、Recovery Manager
阶段 5	优先级与降级执行	priority / deadline、drop policy、degrade mode、circuit breaker
阶段 6	全链路可观测	metrics、logs、traces、live task board、blocked reason 面板

12. 工作时序图

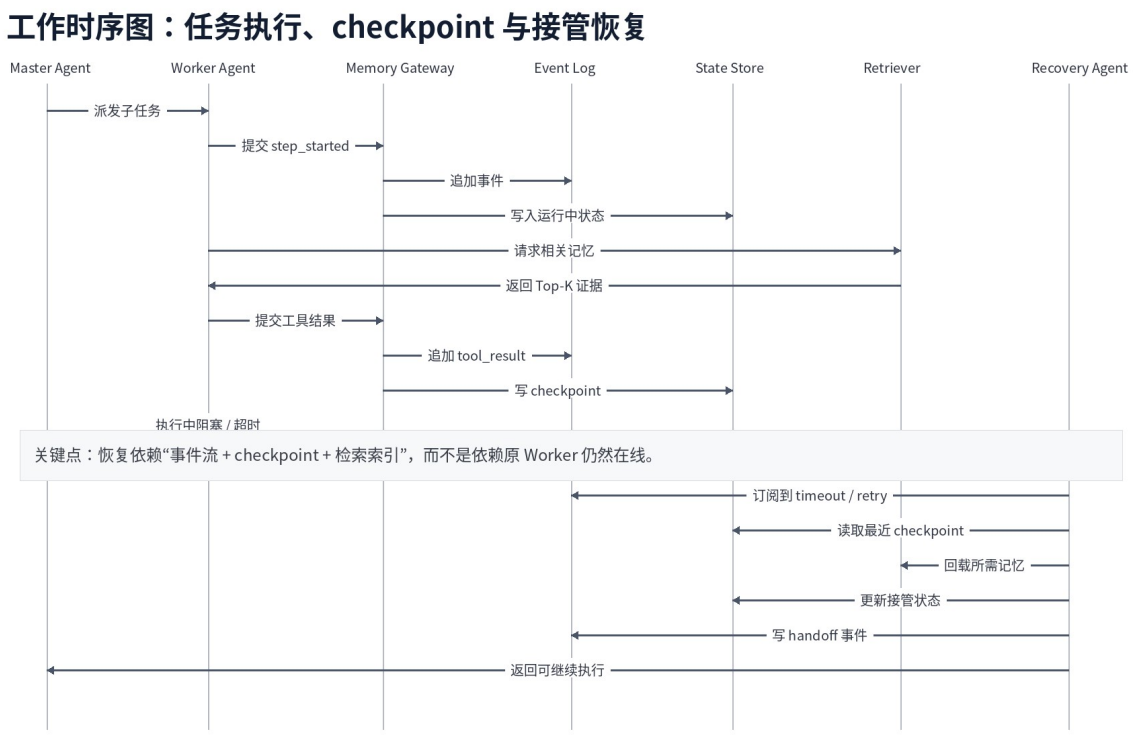


图 4：任务执行、Checkpoint 与接管恢复时序图

- Master Agent 派发子任务给 Worker Agent。
- Worker Agent 通过 Memory Gateway 写入 step_started。
- Gateway 将事件写入事件流，再更新状态层。
- Worker 需要上下文时，Retriever 返回 Top-K 证据。
- Worker 执行过程中持续写入工具结果与 checkpoint。
- 若发生超时或阻塞，Watchdog / Recovery Agent 订阅到异常事件并从最近 checkpoint 接管。

13. 流程图

- 写入流程强调：先记录过程，再落当前真相，再做摘要与索引。
- 回载流程强调：先裁范围，再多路召回，再统一打分，最后压缩成工作记忆。

14. 可观测性与运维

指标	说明
task_latency	任务总耗时
step_latency	步骤耗时

checkpoint_interval	checkpoint 间隔
retrieval_hit_rate	回载命中率
prompt_token_size	L0 工作记忆大小
summary_compression_ratio	摘要压缩率
timeout_count	超时次数
retry_count	重试次数
handoff_count	接管次数
stale_memory_ratio	过期摘要比例

- 建议面板：活跃任务看板、子任务依赖图、阻塞原因分布、最近失败事件流、接管恢复记录、回载质量分析。
- 建议告警：长时间无心跳、task stuck 超过阈值、retry 过多、回载命中率过低、L0 工作记忆大小异常上升、checkpoint 长时间未刷新。

15. 风险与边界

- 不要把热层当唯一真相。
- 不要只做 embedding 检索，必须保留结构化过滤与倒排召回。
- 不要把 checkpoint 做成无法拆解的大黑盒。
- 不要把摘要当原文，必须保留原始引用和溯源关系。
- 不要让所有任务平权，必须从一开始支持 priority / deadline / degrade policy。

16. 最终落地建议

第一版优先闭环： 统一事件模型、状态真相层、checkpoint、摘要器、倒排 + 向量双检索、L0 工作记忆组装器，以及 watchdog + Recovery Manager。做到这一步，系统就从“会聊天的 Agent”升级为“可恢复、可接管、可监控的 Agent Runtime”。

附录：Log + State Store + Retrieval Index + Relation Graph